

Machine Learning

التعلم الآلي

From Stats to ML

Stats:

- Explain the data
- linear regression with statsmodel
- validity of the model with respect to the available data

Machine Learning:

- Predict the data
- linear regression with scikit-learn
- good predictions on new data

From stats to ML

Linear regression with [scikit-learn](#)

LinearRegression

```
class sklearn.linear_model.LinearRegression(*, fit_intercept=True,  
copy_X=True, n_jobs=None, positive=False)
```

[\[source\]](#)

Linear regression with sklearn

Attributes:

coef_ : *array of shape (n_features,) or (n_targets, n_features)*

Estimated coefficients for the linear regression problem. If multiple targets are passed during the fit (y 2D), this is a 2D array of shape (n_targets, n_features), while if only one target is passed, this is a 1D array of length n_features.

- `fit(X, y, sample_weight=None)`
- `predict(X)`
- `score(X, y, sample_weight=None)`

Linear regression with sklearn

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
#  $y = 1 * x_0 + 2 * x_1 + 3$ 
y = np.dot(X, np.array([1, 2])) + 3

reg = LinearRegression().fit(X, y)
reg.score(X, y)

reg.coef_
reg.intercept_

reg.predict(np.array([[3, 5]]))
```

Linear regression with sklearn

On the school dataset

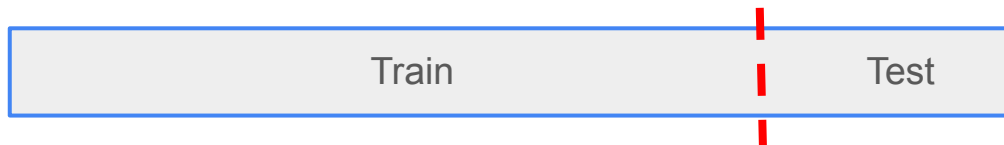
new data ?

ML In a nutshell

- You have a dataset of N samples
- Your goal is to build a model that will make good predictions on new data

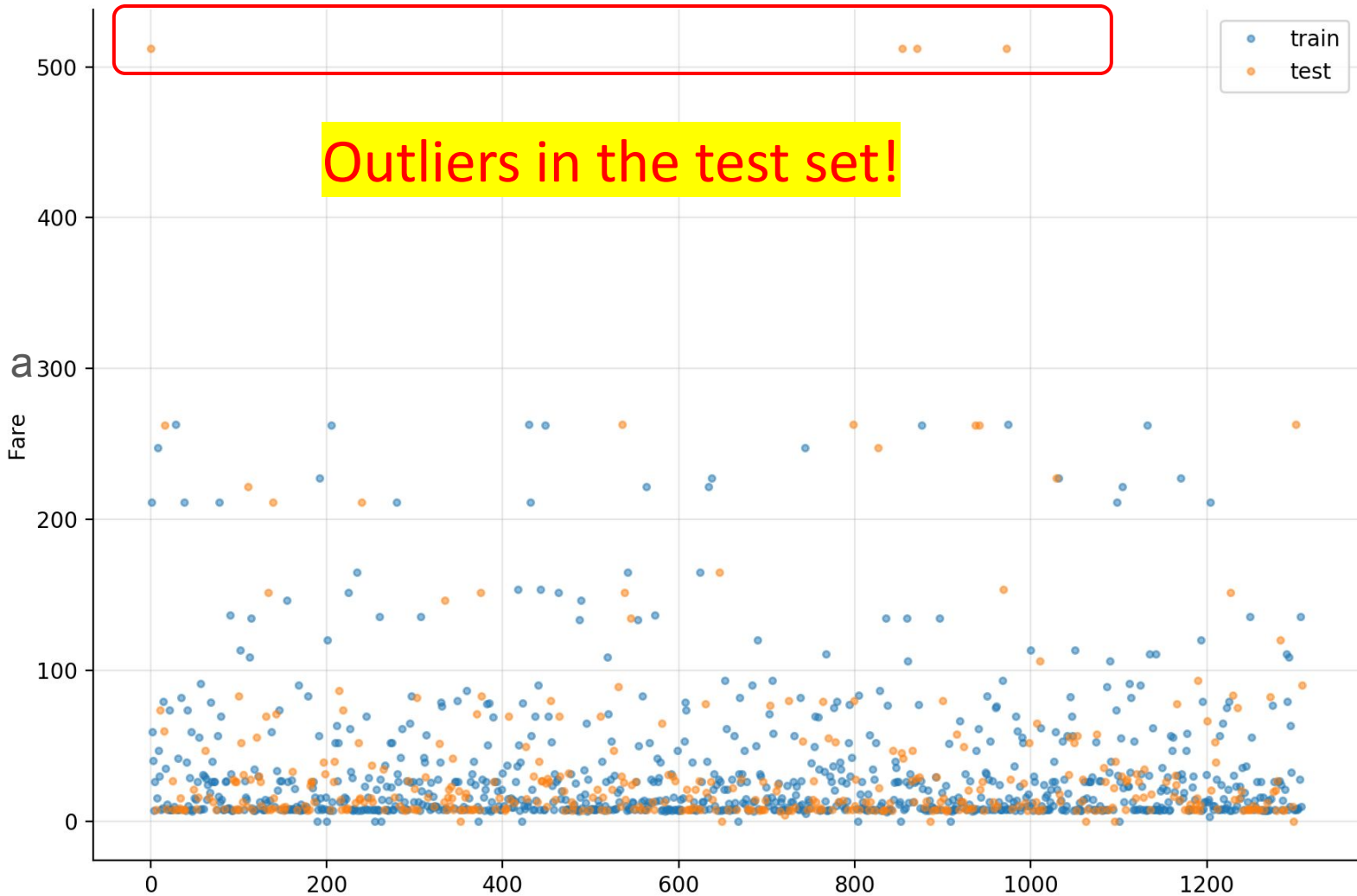
Cross validation:

- You put aside a part of your dataset to act as new data
- You now have a **train** subset and a **test** subset
- You train your model on the train dataset
- You evaluate the model on the test dataset

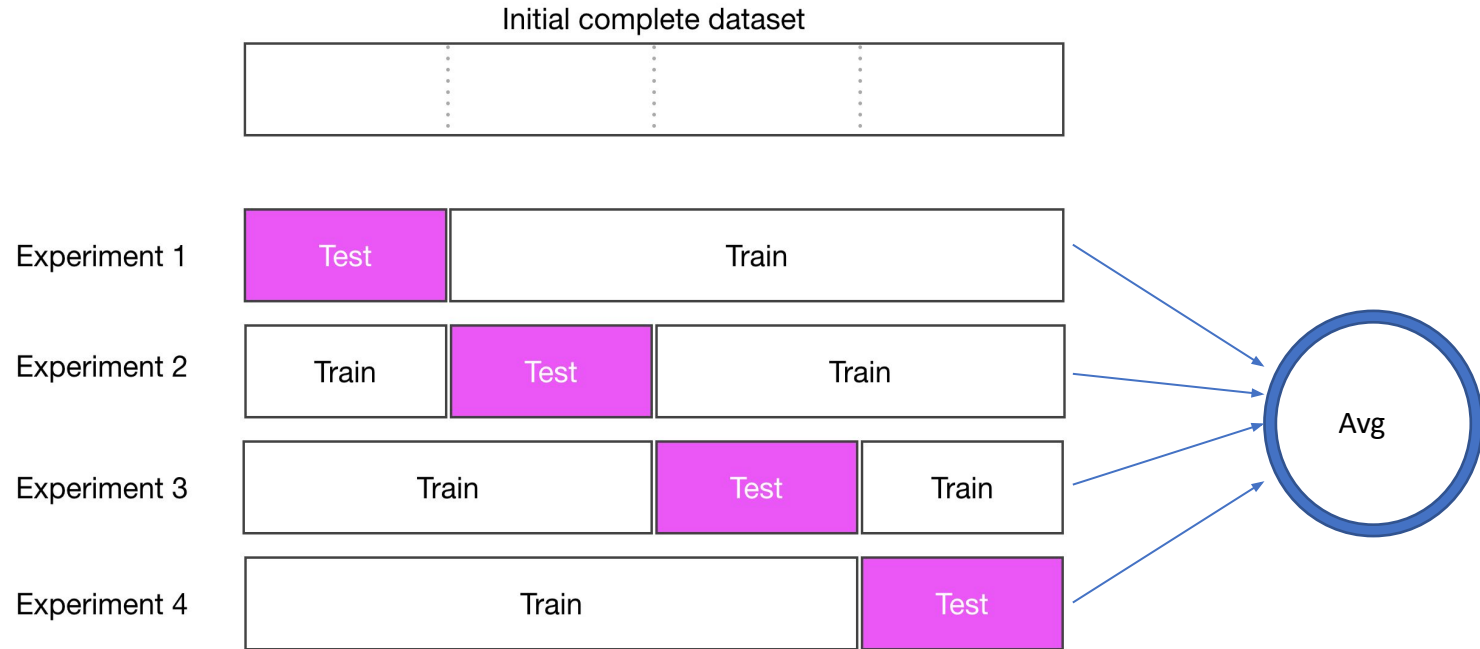


Titanic
single
train test split

you can have a
different
distribution in
both sets



K-fold Cross validation



The model can deal with anomalies
Make the model more robust

Other types of cross validation

- **Stratified K-Fold:** Ensures each fold maintains the same proportion of target classes as the original dataset, **useful for imbalanced data**.
- **Shuffle Split:** Randomly shuffles with replacement and splits data into training and testing sets multiple times, allowing control over test/train sizes.
- **Leave-One-Out (LOO) Cross-Validation:** Uses a single data point as the test set while training on the rest, repeated for all data points.
- **Time Series Split:** Respects temporal order by splitting data into training sets that grow progressively while the test set follows chronologically.

see: https://scikit-learn.org/0.15/modules/classes.html#module-sklearn.cross_validation

Demo - California Housing

```
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()
```

```
In [22]: print(housing.DESCR)
.. _california_housing_dataset:

California Housing dataset
-----

**Data Set Characteristics:**

:Number of Instances: 20640

:Number of Attributes: 8 numeric, predictive attributes and the target

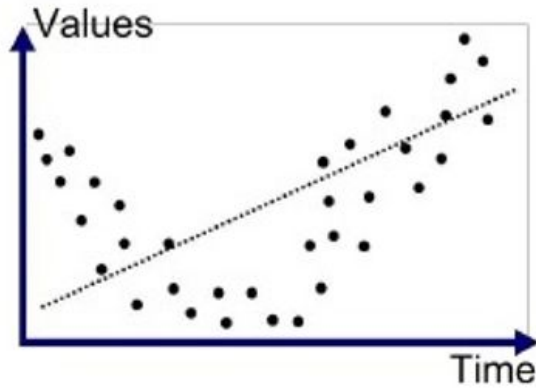
:Attribute Information:
 - MedInc      median income in block group
 - HouseAge    median house age in block group
 - AveRooms    average number of rooms per household
 - AveBedrms   average number of bedrooms per household
 - Population  block group population
 - AveOccup    average number of household members
 - Latitude    block group latitude
 - Longitude   block group longitude
```

<https://colab.research.google.com/drive/1HQsRyd8YxDNcPkpCJgZ7ftlmHBVQca2o>

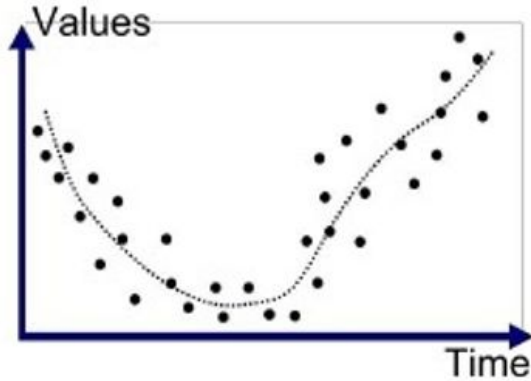
Overfitting !

Overfitting

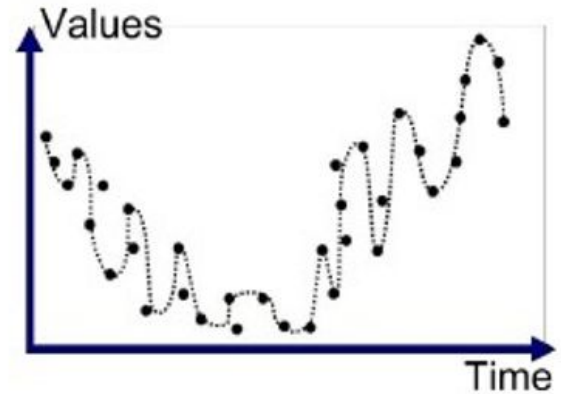
The model learns the data so well that it scores great on the training dataset but poorly on the test dataset.



Underfitted



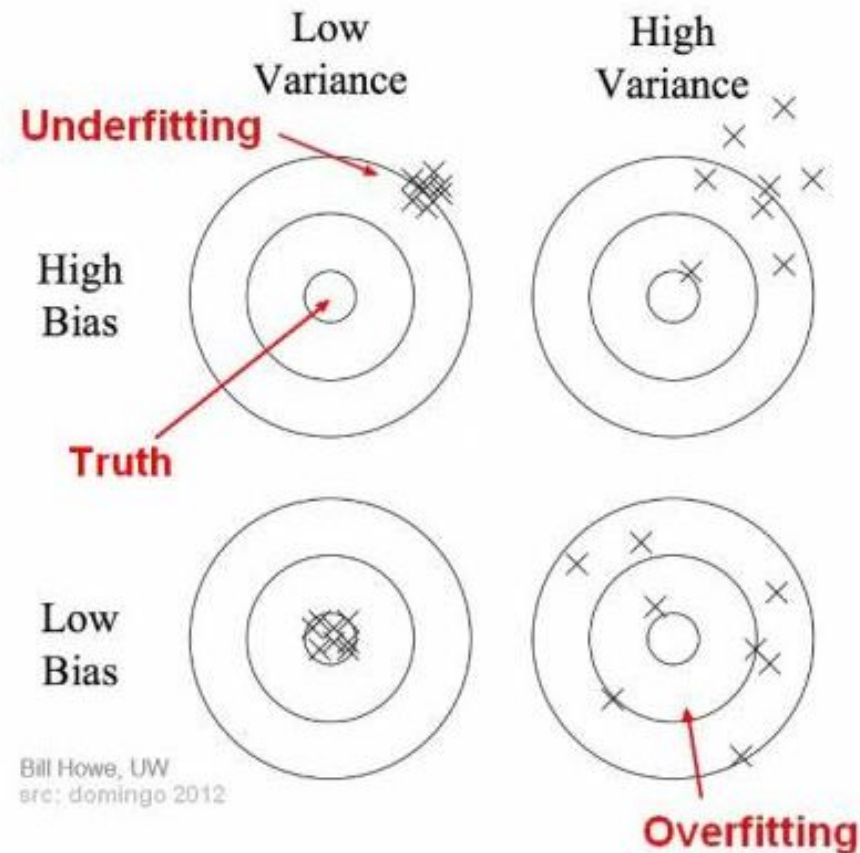
Good Fit/Robust



Overfitted

Overfitting: bias and variance

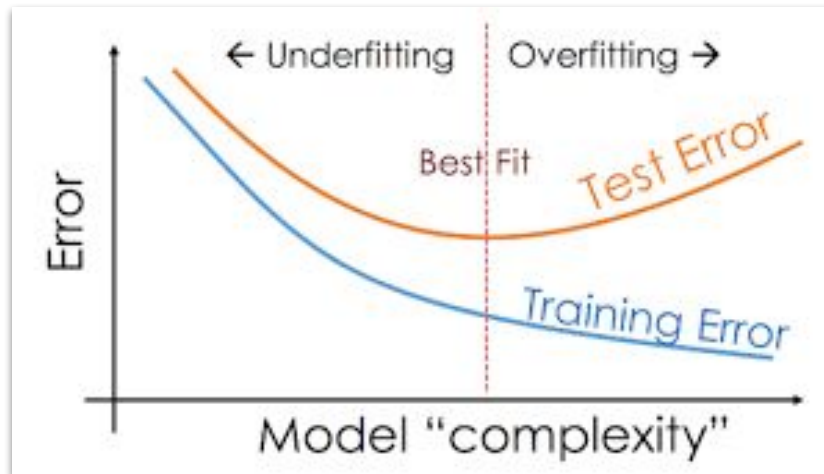
High Bias : misses the target
High Variance : too sensitive to variation in the data (overfits)



Detect Overfitting

Compare the performance on the training and testing dataset

- $\text{score_train} \approx \text{score_test}$
- $\text{score_train} \gg \text{score_test} \Rightarrow \text{overfit}$

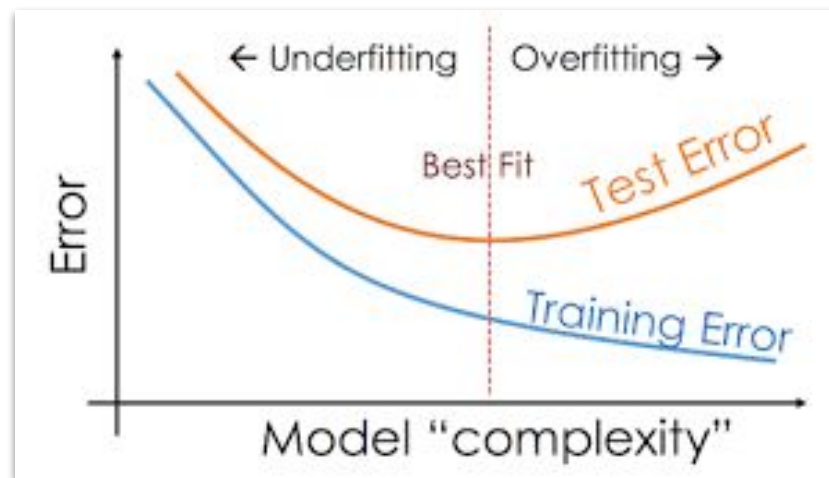


Causes of overfit

- Not enough data to train the model which is equivalent to
- The model is too complex

In both cases, the model learns the training data by heart and cannot extrapolate to new data

Demo => increase training data size and see how both the training error and test error vary



Polynomial regression

Linear regression does not have many parameters.
To illustrate model complexity we can simulate complexity with polynomial regression

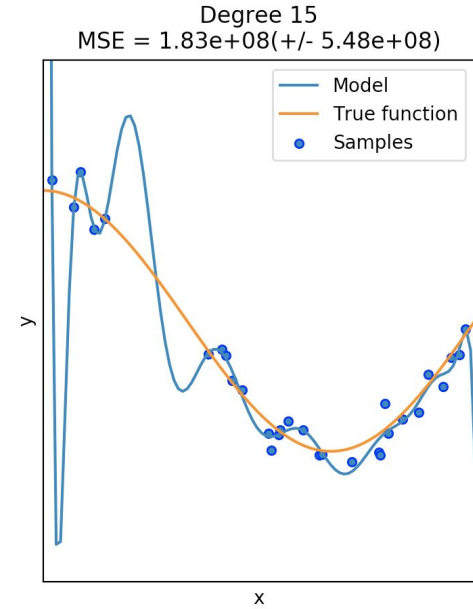
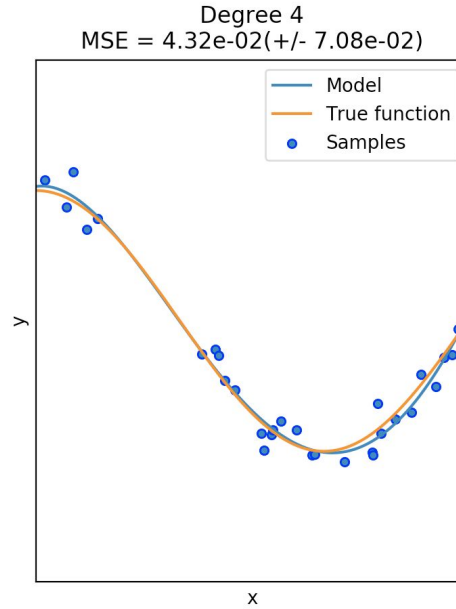
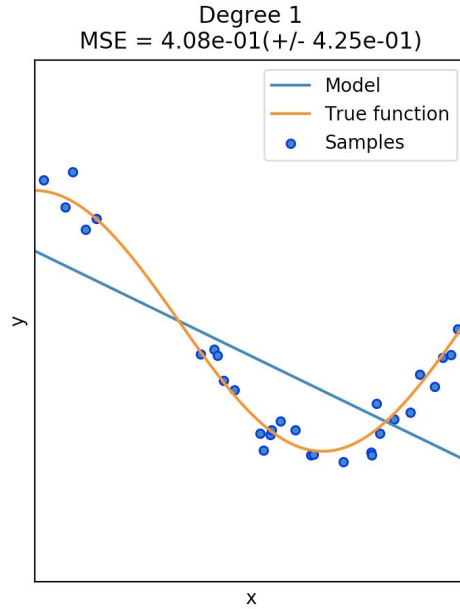
Instead of

$$y \sim a x + b$$

we build

$$y = a_1 x + a_2 x^2 + \dots + a_n x^n + b$$

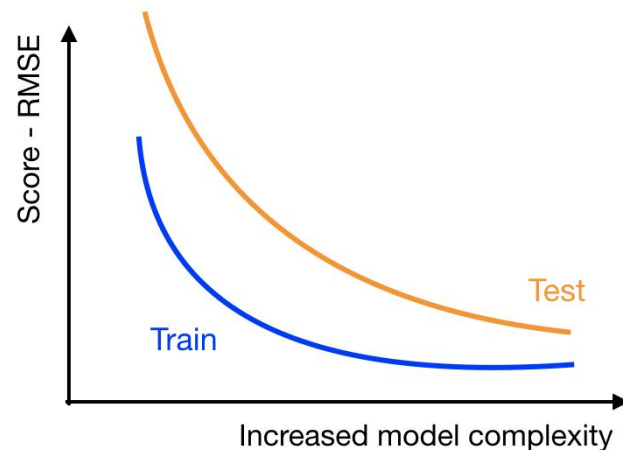
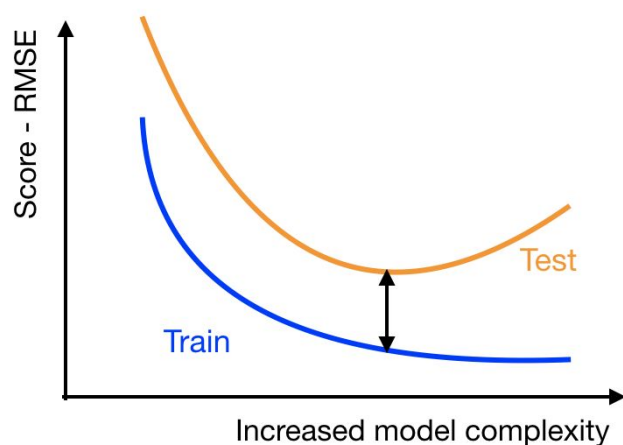
Polynomial regression



https://scikit-learn.org/stable/auto_examples/model_selection/plot_underfitting_overfitting.html

Detector l'overfitting: Learning curve

1. split le dataset en 2 parties: train et test, 70% / 30%
2. calcul de l'erreur de prédiction sur le train et sur le test dataset
 - en augmentant la taille du training set
 - en augmentant la complexité du modèle
3. Learning curves: plot training error vs testing error



A vous – Regression polynomiale - Overfitting

Sur le dataset Ozone,

- split le dataset aléatoirement 70 / 30
- utiliser <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PolynomialFeatures.html>
pour generer des features polynomiales a partir des predicteurs initiaux
- calculer le RMSE en fonction du degré du polynome
 - sur le train subset
 - sur le test subset

Solutions à l'overfitting

Contexte:

- On cherche le(s) paramètre(s) optimum du modèle qui soit le plus robuste
- les modèles trop complexes vont overfitter
- Les modèles trop simples ne prédisent pas bien

Dans la régression non polynomiale, il n'y a pas de paramètres du modèle

Dans la régression polynomiale, l'unique paramètre est le degré du polynôme.

Prévenir l'overfit

- Ajouter des données quand cela est possible
- **Cross validation**: technique de split du dataset pour sélectionner le modèle le plus robuste
- Réduire la complexité du modèle
 - **Régularisation**: modifier la fonction de coût en ajoutant une pénalité liée à la complexité du modèle

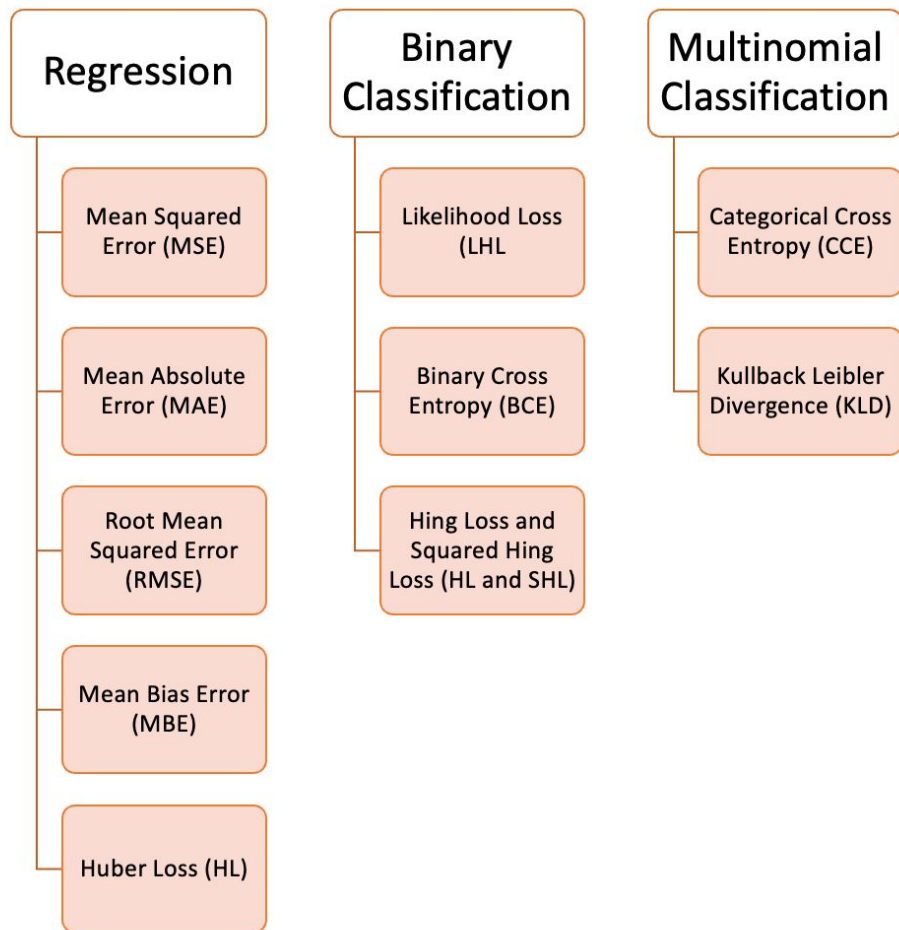
Loss functions

Loss functions

The criteria the model is trying to minimize when training

A function of the error between the true values and the predicted values

$$e = y - y_{\text{hat}}$$



Loss functions - Algo

Change the loss
function

you get a different
algorithm

Model Type	Machine Learning Model	Loss Function
Regression	Linear Regression	Mean Squared Error (MSE)
	Ridge/Lasso Regression	MSE + Regularization (L2/L1)
	Huber Regression	Huber Loss (mix of MSE and MAE)
	Quantile Regression	Quantile Loss
Classification	Logistic Regression	Log Loss (Cross-Entropy)
	Support Vector Machine (SVM)	Hinge Loss
	Neural Networks (Classification)	Cross-Entropy Loss
	Decision Trees / Random Forests	Gini Impurity or Entropy
Unsupervised Learning	K-Means Clustering	Squared Euclidean Distance

Loss functions

L2 norm

L1 norm

Task	Error type	Loss function	Note
Regression	Mean-squared error	$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Easy to learn but sensitive to outliers (MSE, L2 loss)
	Mean absolute error	$\frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	Robust to outliers but not differentiable (MAE, L1 loss)
Classification	Cross entropy = Log loss	$-\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] =$	Quantify the difference between two probability

$$\text{Log Loss} = -\frac{1}{N} \sum_{i=1}^N y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$$

2-Huber Loss

$$l(y_i, \hat{y}_i) = \begin{cases} (y_i - \hat{y}_i)^2 & \text{for } |y_i - \hat{y}_i| \leq \delta \\ 2\delta |y_i - \hat{y}_i| - \delta^2 & \text{otherwise} \end{cases}$$

Regularization

Regularization

Add a constraint on the model to reduce overfitting

contraintes sur le modèle :

- Add a L1 or L2 term on the loss function
- Tune the amount of regularization

L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2}_{\text{Loss function}} + \lambda \underbrace{\sum_{j=0}^M W_j^2}_{\text{Regularization Term}}$$

Loss function with Regularization - general case

Given a set of training examples $(x_1, y_1), \dots, (x_n, y_n), \dots$, $x_i \in \mathbb{R}^m$ $y_i \in \mathbb{R}$ ($y_i \in \{-1, 1\}$ for classification)
our goal is to learn a linear scoring function $f(x) = w^T x + b$
with model parameters $w \in \mathbb{R}^m$ and intercept $b \in \mathbb{R}$

. To find the model parameters, we minimize the regularized training error given by:

$$E(w, b) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \alpha R(w)$$

With

- L a loss function that measures model performance
- R regularisation term that penalizes model complexity (penalty)
- α : positive hyperparameter that controls the regularization strength

Loss functions

Logistic Regression $L(y_i, f(x_i)) = \log(1 + \exp(-y_i f(x_i))).$

OLS $L(y_i, f(x_i)) = \frac{1}{2} (y_i - f(x_i))^2.$

Different regularizations:

Ridge • L2 norm: $R(w) := \frac{1}{2} \sum_{i=1}^n w_i^2,$

Lasso • L1 norm: $R(w) := \sum_{i=1}^n |w_i|$, which leads to sparse solutions.

• Elastic Net: $R(w) := \frac{\rho}{2} \sum_{i=1}^n w_i^2 + (1 - \rho) \sum_{i=1}^n |w_i|$, a convex combination of L2 and L1

Ridge = Linear regression + L2 Regularization

```
class sklearn.linear_model.RidgeClassifier(alpha=1.0, *, fit_intercept=True, normalize=False, copy_X=True,
max_iter=None, tol=0.001, class_weight=None, solver='auto', random_state=None)
```

[\[source\]](#)

Lasso = Linear regression + L1 Regularization

Lasso

```
class sklearn.linear_model.Lasso(alpha=1.0, *, fit_intercept=True,
precompute=False, copy_X=True, max_iter=1000, tol=0.0001, warm_start=False,
positive=False, random_state=None, selection='cyclic')
```

[\[source\]](#)

Linear Model trained with L1 prior as regularizer (aka the Lasso).

The optimization objective for Lasso is:

$$(1 / (2 * n_samples)) * ||y - Xw||^2_2 + \alpha * ||w||_1$$

Ridge Regression

```
class sklearn.linear_model.RidgeClassifier(alpha=1.0, *, fit_intercept=True, normalize=False, copy_X=True,  
max_iter=None, tol=0.001, class_weight=None, solver='auto', random_state=None)
```

[\[source\]](#)

```
sklearn.datasets.make_classification(n_samples=100, n_features=20, *, n_informative=2, n_redundant=2, n_repeated=0,  
n_classes=2, n_clusters_per_class=2, weights=None, flip_y=0.01, class_sep=1.0, hypercube=True, shift=0.0, scale=1.0,  
shuffle=True, random_state=None)
```

[\[source\]](#)

Let's train a Ridge model and look at the influence of the tuning parameter

- Create a binary classification dataset with **make_classification with a noisy dataset** ($n_{\text{informative}} < n_{\text{features}}$)
- Split into train and test using **train_test_split**
- Use RidgeClassifier in scikit-learn on the dataset...
- Vary alpha: 0, 0.01, 0.1, 1, 10, 100
- What do you observe about the model coefficients?
- What do you observe about the model performance when the
- Now try LassoClassifier and assess the impact of the tuning parameter

Lasso Regression

Lasso regularization forces coefficients to zero

Lasso regularization (L1 regularization) forces some coefficients to exactly zero because it uses the L1 norm (sum of absolute values of coefficients) as a penalty in the loss function:

$$\text{Loss} = \text{MSE} + \lambda \sum |w_i|$$

Why does it force coefficients to zero?

The L1 penalty creates a non-differentiable sharp corner at zero, encouraging sparsity.

When optimizing, the model prefers to eliminate less important features by setting their coefficients to zero.

This leads to feature selection, making Lasso useful when dealing with high-dimensional datasets.